289. Game of Life

Medium 1746 277 Add to List Share

According to the Wikipedia's article: "The **Game of Life**, also known simply as **Life**, is a cellular automaton devised by the British mathematician John Horton Conway in 1970."

Given a *board* with *m* by *n* cells, each cell has an initial state *live* (1) or *dead* (0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules (taken from the above Wikipedia article):

1. Any live cell with fewer than two live neighbors dies, as if caused by under-population.
2. Any live cell with two or three live neighbors lives on to the next generation.
3. Any live cell with more than three live neighbors dies, as if by over-population..
4. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

Write a function to compute the next state (after one update) of the board given its current state. The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously.

Example:

```
Input:
[
  [0,1,0],
  [0,0,1],
  [1,1,1],
  [0,0,0]
]
Output:
[
  [0,0,0],
  [1,0,1],
  [0,1,1],
  [0,1,0]
]
```

Follow up:

1. Could you solve it in-place? Remember that the board needs to be updated at the same time: You cannot update some cells first and then use their updated values to update other cells.
2. In this question, we represent the board using a 2D array. In principle, the board is infinite, which would cause problems when the active area

```
class Solution {
    public void gameO
[] board) {
    if (board == null
board.length == 0) re
    int m = board.len
board[0].length;

    for (int i = 0; i
{
        for (int j =
j++) {
            int lives
liveNeighbors(board,
j);

            // In the
every 2nd bit is 0;
            // So we
to care about when wi
bit become 1.
            if (board
&& lives >= 2 && live
                board
// Make the 2nd bit 1
11
            }
            if (board
&& lives == 3) {
                board
// Make the 2nd bit 1
10
            }
        }
    }

    for (int i = 0; i
{
        for (int j =
j++) {
            board[i][
 // Get the 2nd state
        }
    }
}

public int liveNeighb
board, int m, int n,
j) {
    int lives = 0;
    for (int x = Math
0); x <= Math.min(i +
x++) {
        for (int y =
- 1, 0); y <= Math.mi
- 1); y++) {
```

Your previous code was restored from

Console